

SeoulBusMap



서울시 교통 소외 지역을 조명하는 공익 데이터 수집·분석 서비스

중앙대학교 예술공학대학 예술공학부

김도균 (20225525)

김도연 (20232782)

유형준 (20230749)

이지민 (20236743)

Table of Contents

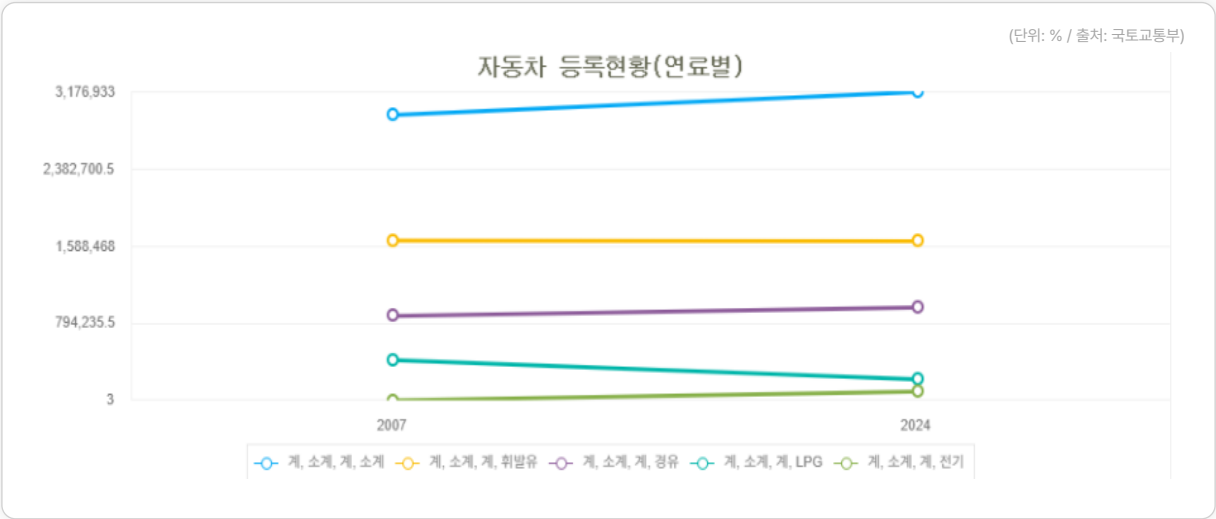
1. 서론
 - 1) 프로젝트의 배경 및 필요성
 - 2) 프로젝트의 개요
2. 본론
 - 1) 시스템 아키텍처: 전체 구조와 설계 철학
 - 2) 데이터베이스 설계: 데이터의 저장과 보존
 - 3) 백엔드 주요 기능: 자동화와 최적화
 - 4) 프론트엔드 주요 기능: 데이터 조회
3. 결론
 - 1) 프로젝트 성과 및 데이터 수집 계획
 - 2) 향후 발전 방향



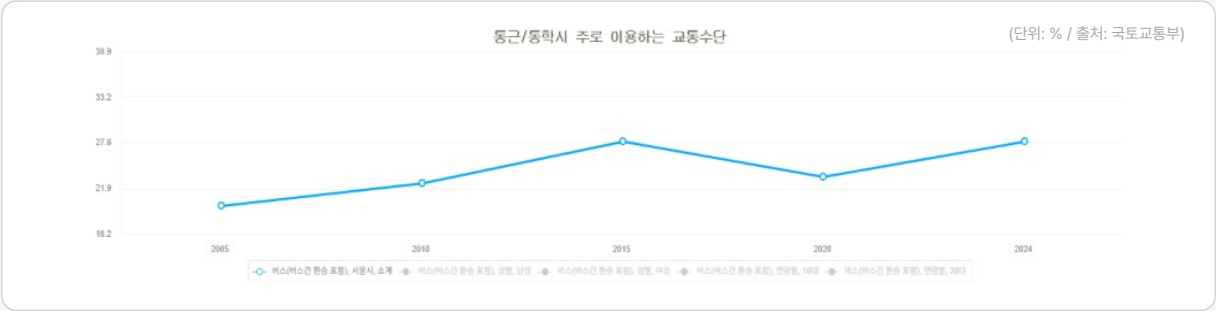
1. 서론

1) 프로젝트의 배경 및 필요성

한국의 대중버스 정책시행은 대구에서 1920년 7월에 시작하여 경성, 현재의 서울에서는 1928년 4월부터 시행되었다. 현재 2025년 약 100년 가까이 대중교통 버스는 시민들과 함께 생활하고 발전해왔다. 한편 대한민국의 자동차 기술 또한 발전하며 자가용(관용, 영업 포함) 자동차를 이용하는 사람들이 늘어났다. 2007년 서울시 자가용 자동차의 수가 2,933,286대를 돌파했고 2024년에는 3,176,933대를 기록했다.



이러한 통계를 바탕으로 자가용 자동차 차량 이용이 증가한 만큼 교통버스의 이용률이 줄었을 것이라고 추측하기 쉽지만 그렇지 않다. 국토교통부 통계청에 따르면 통근/통학시 주로 이용하는 이동수단을 조사했을 때 2005년에는 19.7%의 사람이 버스를 이용하는 반면 2024년에는 27.7%로 오히려 8% 증가한 모습을 보여준다.



위 그래프까지 하여 확인할 수 있듯이, 자가용의 확대에도 불구하고 대중교통 이용은 서울 시민들의 도시 생활의 필수 인프라로 곳곳이 그 자리를 지키고 있다. 그러나 '이러한 혜택이 모든 지역에 균등하게 돌아가고 있는가?' 라는 질문에는 의문점이 든다. 서울은 대한민국의 수도로서 천만 인구가 밀집된 거대 도시이지만, 지역별 인구 분포와 밀집도에는 현저한 차이가 존재한다. 특정 행정동에는 수만 명의 인구가 거주하는 반면, 다른 행정동은 그보다 훨씬 적은 인구를 가지고 있다. 이러한 인구 분포의 차이는 곧 대중교통 서비스에 대한 '수요'의 차이로 이어진다. 시민 생활의 필수재인 대중교통 서비스가 이러한 수요에 맞춰 적절하게 '공급'되고 있는지, 즉 인구 밀집 지역과 인구 희소 지역 간에 서비스 수준의 불균형은 없는지 확인할 필요가 있다. 만약 인구는 많지만 버스 노선이나 정류장 수가 부족한 지역이 있다면, 해당 지역 주민들은 다른 지역 주민들에 비해 명백한 불편을 겪게 될 것이다. 본 프로젝트에서는 이렇게 세운 가설을 바탕으로, 서울시 행정동별 거주 인구 데이터와 해당 지역에 공급되는 버스 서비스(정류장, 노선 수 등) 데이터를 결합하여 '수요 대비 공급의 적정성'을 심층적으로 분석하고자 한다. 이를 통해 데이터 상으로 나타나는 지역 간 대중교통 서비스의 격차를 식별하고, 이러한 격차가 일부 지역 주민에게 '교통 소외'라는 현실적인 불편을 야기하며, 나아가 지역 간 삶의 질 불균형 문제로 직결될 수 있음을 실증적으로 보여주려 한다.

2) 프로젝트의 개요

이러한 생각의 흐름으로 세운 가설을 데이터 기반으로 검증하고, 교통 소외 문제를 실증적으로 조명하기 위해 본 프로젝트에서는 조각조각 분산되어 복잡한 구조로 제공되는 행정동 인구와 버스 정류장 현황 데이터를 수집, 정제, 보존하여 최종적으로 의미 있는 정보로 시각화하는 하나의 완성된 서버 애플리케이션 구축을 목표로 했다. 여기에 더해 단순히 가설 검증만으로 기능을 다하는 것이 아닌, 시간의 흐름에 따라 변화하는 도시의 교통 환경을 지속적으로 추적하여 지속적으로 새로운 의미를 창출해낼 수 있는 서비스가 될 수 있도록 설계해 보고자 한다. 이러한 목표를 가지고, 중간고사 팀 프로젝트의 요구 사항 범위에 맞춰 본 보고서에서는 데이터 수집 및 저장 단계까지 다루어 보았다. 먼저, 문제 해결을 위한 서비스의 주요 기능 구현 중점은 다음과 같이 정하였다:

1. 분산된 데이터의 통합 수집:

가설 검증의 기반이 되는 ‘수요’ 지표(행정동별 생활 인구 등)와 ‘공급’ 지표(버스 정류장 위치, 노선 정보, 승하차 인원 등)은 각기 다른 형태로 여러 공공 API에 흩어져 있다. 본 프로젝트에서는 이러한 다종의 API들을 안정적으로 호출하고, 각기 다른 형식의 데이터를 일관된 구조로 통합 수집하는 파이프라인을 구축한다.

2. 다양한 목적에서의 사용을 고려한 데이터 보존:

서울시에서 제공하는 공공 API는 대부분이 한 달 안팎의 데이터 보존 기간이 명시되어 있다. 장기적인 관점에서의 분석에 사용하기엔 턱없이 짧은 기한이다. 데이터의 보존을 위해 수집된 데이터를 적절하게 보존할 필요가 있다. 그러나 보존 기간이 늘어날수록 불가피하게 커지는 용량과 느려지는 처리 속도를 고려, 사용자의 요청에 대응하기에 부족하지 않으면서도 필요한 데이터와 필요하지 않은 데이터를 분리하여 보존하고 서비스할 수 있도록 구현한다.

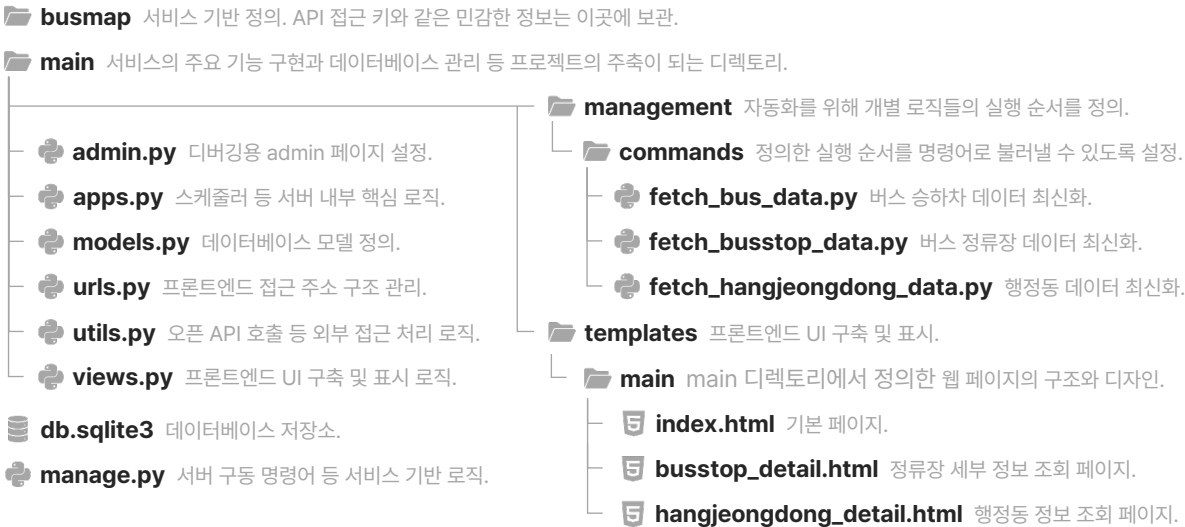
3. 지속 가능한 수집과 분석을 위한 운영 환경 구축:

아무리 현 상황에 맞는 데이터를 잘 수집하고 분석하더라도 일회성 분석은 시간의 흐름에 따라 세상이 바뀌며 의미를 잃는다. 본 프로젝트에서는 시간이 지나가더라도 그에 맞춰 따라가며 최신의 정보를 제공하면서도 과거와의 추이 변화를 분석할 수 있도록 수집과 보존이 가능한 자동화로 이루어질 수 있도록 하는 시스템을 지향한다.

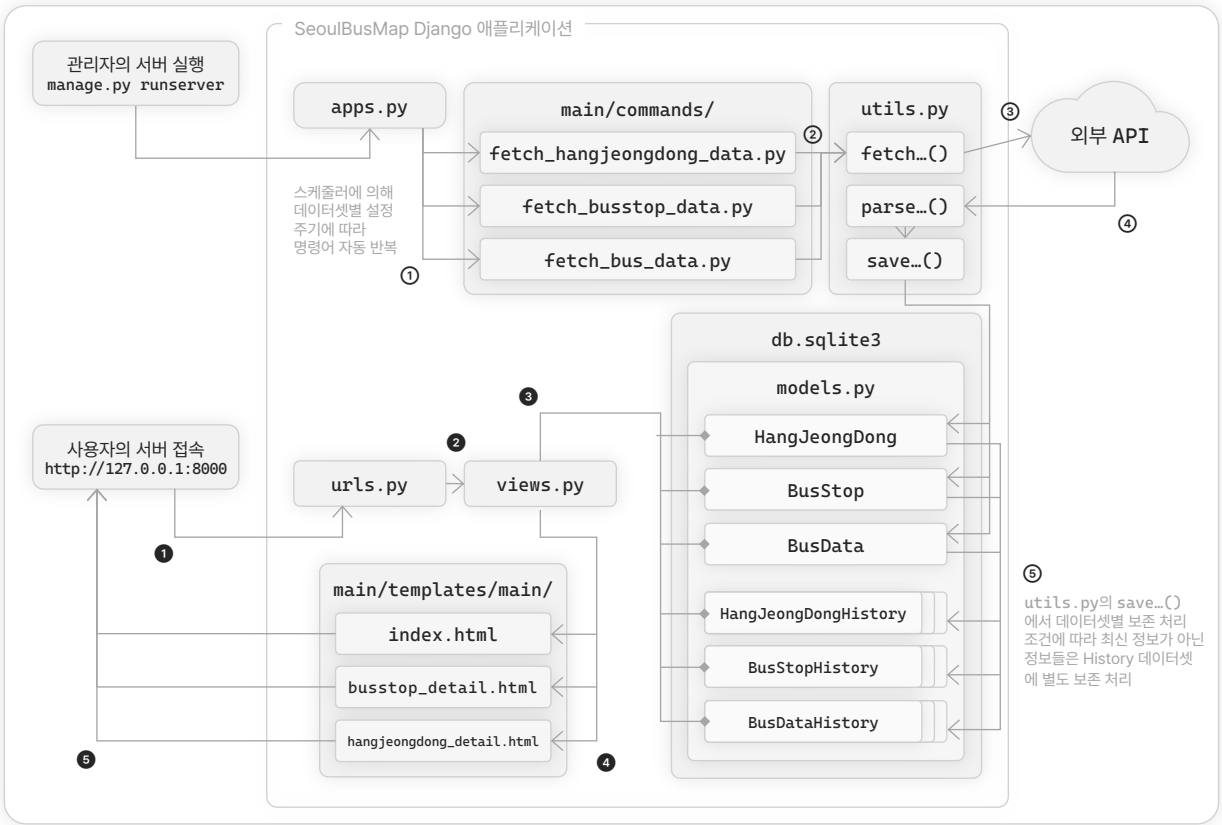
2. 본론

1) 시스템 아키텍처: 전체 구조와 설계 철학

강의에서 실습하며 제작한 샘플 서버 프로젝트를 바탕으로, 본 프로젝트의 토대를 만들어 개발을 진행하였다. Django 시스템 파일이나 기본 제공되는 미사용 파일을 제외한 핵심 기능을 구성하는 전체 구조와 파일·모듈별 기능은 다음과 같다:



이렇게 정의내린 각각의 모듈들이 서로 상호작용하며 기능을 실행하는 전체적인 흐름도는 아래 그림과 같다. Django 설계 철학인 MVT(Model-View-Template) 패턴을 충실히 따르려 노력하면서도, 서버의 효율성과 가독성을 고려하여 구동 방식을 설계하였다:



데이터 수집 및 아카이빙의 흐름 (백엔드)

- ① apps.py → commands/: 작업 스케줄러가 주기적으로 특정 관리자 명령어를 실행한다.
- ② commands/ → utils.py: 명령어에 설계된 순서와 작업하려는 데이터 모델별 인자를 적용하여 유틸리티 함수를 실행한다.
- ③ utils.py → 외부 API: fetch() 함수로 주어진 인자에 맞는 외부 API로부터 데이터를 요청하고 응답받는다.
- ④ 외부 API → utils.py: parse() 함수로 응답받은 데이터를 DB에 저장할 수 있도록 필요한 정보만 거르고 형식을 맞춘다.
- ⑤ utils.py → 데이터베이스: save() 함수로 최신 데이터를 저장하고, 조건에 맞다면 보존용 모델에 데이터를 저장한다.

사용자 요청의 처리 흐름 (프론트엔드)

- ① 사용자 → urls.py: 사용자가 웹 브라우저를 통해 서버의 URL에 접속을 요청한다.
- ② urls.py → views.py: urls.py에 정의된 라우팅 규칙에 따라 해당 views.py의 함수가 호출된다.
- ③ views.py ↔ 데이터베이스: views.py가 데이터베이스에서 사용자 요청에 필요한 데이터를 조회하고 응답받는다.
- ④ views.py → templates/: views.py가 가공된 데이터를 templates/의 HTML파일로 전달하여 렌더링을 요청한다.
- ⑤ templates/ → 사용자: 렌더링된 HTML 페이지가 웹 브라우저를 통해 사용자에게 최종적으로 표시된다.

위의 다이어그램과 순서도별 절차가 우리 시스템의 전체적인 구조와 데이터 흐름을 개괄적으로 나타낸 것이다. 이제부터는 프로젝트의 세 가지 기둥이 되는 주요 구성 요소인 다음 세 가지 범주로 프로젝트를 구성하는 파일과 모듈을 나누고:

- 데이터베이스 설계 (models.py, utils.py)
- 백엔드 (admin.py, apps.py, management/)
- 프론트엔드 (urls.py, views.py, templates/)

나누어 둔 범주에 해당하는 모듈의 구체적인 구현 내용을 훑아보고, 개발 과정에서 생긴 고민과 그것을 해결한 방안을 소개하며 그 안에 담긴 설계 의도를 상세히 설명하고자 한다.

2) 데이터베이스의 설계: 데이터의 저장과 보존

전술했듯, 본 프로젝트의 핵심 목표는 흩어져 있는 행정동 인구 수와 버스 정류장 위치 정보, 정류장별 승하차 인원 수와 같은 대중교통 이용 추이를 분석할 수 있는 데이터를 수집하고, 시간의 흐름에 따른 변화를 추적할 수 있도록 보존하여 교통 소외 지역의 분석을 할 수 있도록 제공하는 것이다. 이러한 목표를 달성하기 위해 다음 3개의 데이터 모델을 `models.py`에 설계하였다:

- **HangJeongDong**: 이 모델은 분석의 주체인 '수요'를 정의한다. 서울시 각 행정동의 고유 ID와 이름, 그리고 생활 인구 API를 통해 수집된 일 평균 인구 수 정보를 저장하여 지역별 수요의 크기를 얻는다.
- **BusStop**: 이 모델은 본 프로젝트에서 대중교통 '공급'의 단위로 삼고자 하는 버스 정류장의 물리적 정보를 담는다. 각 정류장의 ID, 명칭, 그리고 지도 시각화와 주소 획득에 얻을 수 있는 좌표를 저장하여 공급 인프라의 분포를 파악하는 기준을 얻는다.
- **BusData**: 이 모델은 '공급'이 실제 수요로 이어져 이용되는 현황을 파악하기 위한 데이터이다. 특정 날짜에 노선별 정류장에서 기록한 일일 총 승하차 인원 정보를 저장하여, 공급되고 있는 서비스가 실제로 얼마나 이용되는지를 측정하는 지표를 얻는다.

각 모델별 세부 데이터 필드 구조와 필드별 수집 목적 및 사용 방안, 그리고 정보 출처와 저장 절차는 다음과 같다:

2-1) HangJeongDong

HangJeongDong

서울시 행정동명과 거주 인구 수를 저장하는 모델

district_id

11590605

CharField

하나의 행정동 ID.
(최대 길이 10자, 중복 불가)

name

흑석동

CharField

하나의 행정동 이름.
(최대 길이 100자, 중복 불가)

population

33201

IntegerField

하나의 행정동 총 생활 인구 수.
(빈 값 가능)

timestamp

2025-10-30 00:00

DateTimeField

데이터가 수집된 기준 시간.
(날짜/시간 형식)

수집 목적 및 사용 방안

행정동별 인구 수에 대비한 데이터 분석 결과를 도출하기 위해 받아오는 데이터이다. 자주 큰 폭으로 바뀌는 데이터는 아니지만, 인구 수 필드는 어느 정도 시기에 따른 정확도는 필요한 데이터라고 판단하였다. 장기적 관점에서 이뤄지는 교통 정책에 대응할 수 있도록 비교적 넓은 간격(한 달 등)으로도 데이터를 새로 받아 최신화 할 소요가 있다.

district_id	행정동별 데이터의 구분을 위한 ID를 담는 필드. 트리 구조 형성에 사용.
name	사용자에게 출력할 때를 위해, 행정동별 ID에 따른 한글명을 담는 필드.
population	인구 수에 대비한 분석을 위해 모으는 인구 수를 담는 필드.
timestamp	일자별 조회와 분석을 위해 수집 시각을 기록하는 필드.

정보 출처와 저장 절차

서울시 열린데이터광장에서 제공하는 '서울시 상권분석서비스(영역-행정동)' API에서 행정동 식별 ID와 그에 대응하는 한글 행정동명을 받아오고, 같은 출처에서 제공하는 '행정동 단위 서울 생활인구(내국인)' API에서 행정동 식별 ID별 생활인구수를 받아온다.

'fetch_hangeongdong_data.py'에서 API 요청과 DB 저장을 관리한다. '행정동 단위 서울 생활인구 (내국인)'에서는 식별 ID와 생활인구수 관련 데이터만 제공하기 때문에, 부득이 두 가지의 API 요청 결과를 조합하여 사용하는 형태로 DB에 저장하는 구조를 취하게 되었다. 우선 '서울시 상권분석서비스(영역-행정동)'에서 현재 존재하는 행정동의 식별 ID와 그에 대응하는 한글명을 받아온다. 그런 다음, '행정동 단위 서울 생활인구(내국인)'에서 행정동의 식별 ID별 평균 인구수를 받아온다. 해당 데이터는 매일 최신화가 명세되어 있지만, 조회 시간에 따라 해당 날짜의 최신 데이터가 아직 올라오지 않는 경우가 있었다. 그래서, 현재 날짜로부터 과거 7일간의 데이터 존재 여부를 확인하고, 데이터가 있는 가장 최근 날짜의 정보를 받아온다. 해당 API에서는 24시간 시간별로 인구 수 데이터를 개별적으로 제공하기 때문에, 취합을 위해 하루 간의 평균 인구수를 계산하여 최종적으로 해당 데이터를 DB에 저장하게 된다. 24시간 × 400여 곳의 행정동 정보를 받아오기 때문에 만 건이 넘는 자료의 요청을 보내야 하지만, 서울시 API에서는 한 번에 1,000건까지의 요청만 접수하는데, 그래서 API 자체적으로 제공해 주는 전체 데이터 개수를 파악한 후, 페이지네이션 기법을 통해 1,000건씩 나누어 요청을 반복하여 보내는 식으로 처리하였다.

2-2) BusStop

BusStop

버스 정류장의 기본 정보를 저장하는 모델

busstop_id "119900101"
CharField

하나의 정류장 고유 ID.
(최대 길이 20자, 중복 불가)

name "중앙대정문"
CharField

하나의 정류장 고유의 이름.
(최대 길이 100자)

longitude 126.9582940
DecimalField

하나의 정류장의 지도상 경도.
(최대 길이 10자 중 소수점 7자)

latitude 37.5068110
DecimalField

하나의 정류장의 지도상 위도.
(최대 길이 10자 중 소수점 7자)

district_id "11590605"
CharField

정류장이 속한 행정동 ID.
(최대 길이 10자, 빈 값 가능)

is_active True
BooleanField

정류장의 활성화 여부.
(최대 길이 10자, 빈 값 가능)

timestamp 2025-10-30 00:00
DateTimeField

데이터가 수집된 기준 시간.
(날짜/시간 형식)

수집 목적 및 사용 방안

어느 버스 정류장이 어디 위치에 있는지를 받아오는 데이터이다. 행정동별 인구 수와 마찬가지로 자주 큰 폭으로 바뀌는 데이터는 아니지만, 교통 정책의 변화에 따라 비교적 넓은 간격의 주기로 최신화해 줄 필요가 있는 데이터이다.

busstop_id	버스 정류장별 데이터의 구분을 위한 ID를 담는 필드. 트리 구조 형성에 사용.
name	사용자에게 출력할 때를 위해, 정류장 ID에 따른 한글명을 담는 필드.
longitude	지도상 위치와 소속 행정동을 찾기 위한 경도 좌표 필드.
latitude	지도상 위치와 소속 행정동을 찾기 위한 위도 좌표 필드.
district_id	소속 행정동별 데이터의 구분을 위한 ID를 담는 필드. 트리 구조 형성에 사용.
is_active	현재 사용되고 있는 정류장인지 여부를 구분하는 필드.
timestamp	일자별 조회와 분석을 위해 수집 시각을 기록하는 필드.

정보 출처와 저장 절차

서울시 열린데이터광장에서 제공하는 '서울시 버스정류소 위치정보' API에서 정류장 고유 ID와 그에 대응하는 한글명, 지도상 위치를 받아온다. 카카오에서 제공하는 '카카오맵 로컬 REST API'를 활용하여 버스 정류장이 위치한 좌표가 어느 행정동에 속하여 있는지를 받아온다.

'fetch_busstop_data.py'에서 API 요청과 DB 저장을 관리한다. 행정동별 인구 수 데이터와 마찬가지로 서울시 전역의 만 개가 넘는 정류장별 승하차 인원수의 데이터를 받아와야 하지만, 한번에 1,000건 까지의 제한이 있기에, 마찬가지로 페이지네이션을 활용하여 1,000건씩 나누어 요청을 보낸다. 서울시 공공데이터에서는 기본적으로 특정한 버스 정류장이 위치한 지도상의 좌표만 제공하고, 주소나 소속된 행정동 정보는 별도로 제공하지 않기에, 제공되는 좌표를 통해 이 정류장이 어느 행정동에 위치하여 있는지를 확인하는 과정에서 '카카오맵 로컬 REST API'에서 제공하는 좌표의 행정동 주소로의 변환 서비스를 거쳐 최종 데이터를 수집하게 된다. 또, 교통 정책에 따라 생겼다 사라질 수도 있는 버스 정류장이기에, 정류장이 사용되고 있는지, 아닌지 상태를 별도로 추적한다. 사용되고 있지 않은 상태(폐지된 정류장)이라도 과거 데이터를 열람하여 시간에 따른 추이를 확인할 수 있게 하고자 'is_active' 데이터로 정류장 활성 여부를 추적하여 데이터를 삭제하지 않고 보존한다.

버스 정류장 기초 데이터에 활성화/비활성화 여부를 별도 필드로 저장하는 것이 필요한 이유는, 버스 정류장은 현실에서 임시로 운영이 중단되는 등 상태가 변할 수 있다. 서울시 API에서는 이런 정류장을 단순히 목록에서 제외시키는 방식으로 반영한다. 비활성 정류장을 보존하지 않고 삭제한다면, 과거 정보를 조회하는 데에 추가적인 계산이 필요할 수 있고, 나중에 다시 운영을 시작하여 API에 재등장한다면 저장된 데이터의 타임라인이 중간에 불필요하게 끊기게 된다. 정류장의 이용자가 없으면 비활성 상태인 식으로 논리적으로 처리하는 방법도 고려할 수 있으나, 정말 운영 중인데 이용자가 0명이거나, 공사, 시위나 명절 연휴 등의 사유로 노선이 임시 변경되는 등의 End-Case에서 더 나은 대안이 없는 것으로 보였다. 추가로, 행정동의 경우 변경 주기가 더욱 드물고 해당 데이터는 위치나 주소 데이터보다 인구 수의 변동 추이가 더욱 중점인 데이터였기에 정류장 데이터만이 해당 필드를 가지게 설계하였다.

2-3) BusData

BusData

버스 정류장의 승하차 인원수를 저장하는 모델

busstop_id "119900101"
CharField

하나의 정류장 고유 ID.
(최대 길이 20자, 중복 가능)

bus_id "11110602"
CharField

하나의 버스 노선 고유 ID.
(최대 길이 20자, 중복 가능)

passenger_off 954
IntegerField

해당 정류장의 하루간 하차 인원.
(정수 형식)

passenger_on 134
IntegerField

해당 정류장의 하루간 승차 인원.
(정수 형식)

timestamp 2025-10-30 00:00
DateTimeField

데이터가 수집된 기준 시간.
(날짜/시간 형식)

수집 목적 및 사용 방안

버스 정류장별 승하차 인원수를 받아오는 데이터이다. 본 프로젝트 주제의 골자가 되는 중요한 데이터로, 매일매일 서울시 버스 교통 이용 현황을 '노선'과 '정류장'이라는 가장 세분화된 단위로 파악하고, 이를 통해 교통 접근성을 분석하기 위한 원천 데이터가 된다.

busstop_id 버스 정류장별 데이터의 구분을 위한 ID를 담은 필드. 트리 구조 형성에 사용.
bus_id 해당 데이터의 출처인 버스 노선의 구분을 위한 ID를 담은 필드.
passengers_on 하루 동안 정류장에서 해당 노선에서 몇 명이 승차했는지를 담은 필드.
passengers_off 하루 동안 정류장에서 해당 노선에서 몇 명이 하차했는지를 담은 필드.
timestamp 일자별 조회와 분석을 위해 수집 시각을 기록하는 필드.

정보 출처와 저장 절차

서울시 열린데이터광장에서 제공하는 '서울시 버스노선별 정류장별 승하차 인원 정보' API에서 정류장 고유 ID와 버스 노선 ID, 기준 날짜와 실제 승하차 인원 데이터를 받아온다.

'fetch_bus_data.py'에서 API 요청과 DB 저장을 관리한다. 행정동별 인구 수나 버스 정류장 기초 데이터와 마찬가지로 버스 노선 × 정류장 개수로 4만 개가 넘는 정류장의 데이터를 받아와야 하지만, 한 번에 1,000건 까지의 제한이 있기에, 마찬가지로 페이지네이션을 활용하여 1,000건씩 나누어 요청을 보낸다. 해당 데이터는 매일 최신화가 명세되어 있지만, 조회 시간에 따라 해당 날짜의 최신 데이터가 아직 올라오지 않는 경우가 있었다. 그래서, 현재 날짜로부터 과거 7일간의 데이터 존재 여부를 확인하고, 데이터가 있는 가장 최근 날짜의 정보를 받아온다. 매일매일 달라지는 수치가 보장된 변동성이 높은 데이터이기 때문에, API 업데이트가 하루 이틀 정도 지연되는 경우에도 문제없이 데이터를 수집할 수 있도록 안정성을 고려하여 이러한 수집 방법을 채택하였다. 몇 만 개의 단위가 넘어가는 데이터를 수집하다 보니, 기존 실습에서 다루었던 `.save()`나 `.create()` 메서드의 처리 시간 지연이 두드러졌다. 한 번에 하나의 데이터를 저장하거나 처리하다 보니 데이터베이스 접근 횟수도 만 회 단위가 넘어가며 생긴 문제로, 큰 데이터를 다루기 위한 더 나은 방법으로 `.bulk_create()` 사용을 채택하였다. 수집한 데이터를 임시로 메모리에 저장한 다음, 데이터베이스와는 한 번의 접근만으로 한꺼번에 데이터를 이동시켜 병목을 최소화하는 기법이다. 일부 자동화 로직을 사용할 수 없다는 단점이 있는 것으로 보였지만, 본 프로젝트에서 요구하는 사항에 제약은 없는 것으로 판단하여 본 데이터 모델과 함께 HangJeongDong과 BusStop 데이터를 수집할 때에도 이 메서드를 사용하였다.

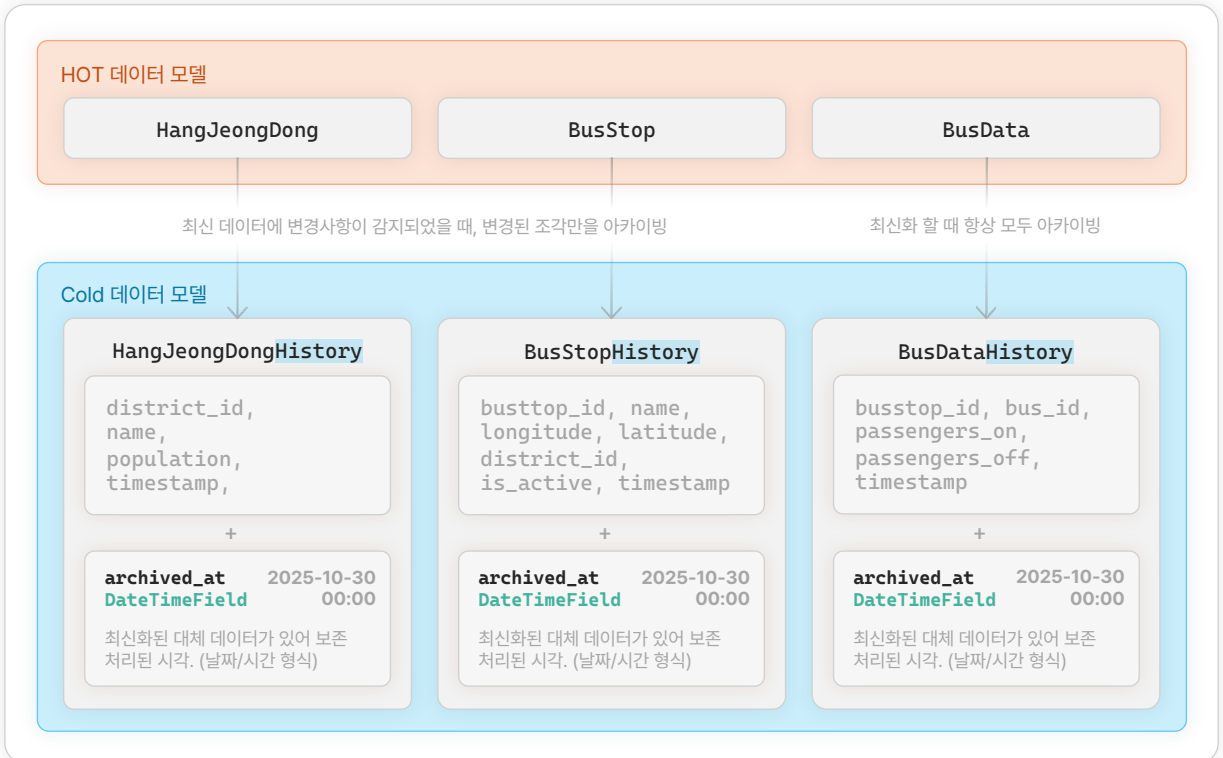
행정동과 정류장 기초 데이터는 최신화 빈도가 적어 timestamp를 서버 시간 기준으로 해도 무관하였지만, 일일 데이터는 야간 버스에서 자정을 넘어가는 경우 등에서 기준 일자와 저장 시간대가 불일치하는 경우가 있었다. 그래서 본 데이터에서의 timestamp는 서울시에서 제공하는 날짜 데이터로 채운다.

2-4) 수집한 데이터를 보존하는 방안

시간이 갈수록 규모가 커질 수밖에 없는 데이터셋을 관리하고 분석하여야 하는 본 서비스의 특성 상, 서버 용량과 처리량에 관한 고려는 필연적으로 거쳐가야 하는 관문이었다. 이러한 맥락에서 본 프로젝트의 핵심 주제인 '교통 소외 지역 분석'의 기능을 수행하기 위하여 사용자의 입장에서 이 서비스에 접근하는 동기를 고려했을 때, 사용자의 이용 목적은 크게 두 가지 흐름으로 귀결할 수 있었다. 첫 번째는 '① 현재 당장 내 지역의 대중교통 접근성'이 궁금한 경우로, 지금 우리 동네의 버스 이용은 얼마나 편리한지, 이사를 고려하는 사람들이 지역을 선택할 때 교통 접근성을 어떻게 판단해야 할지와 같은 빠르고 일상적인 의사결정에 영향을 미치는 기능이다. 두 번째는 '② 과거를 포함한 역대 대중교통 접근성의 변화 이력'을 확인하고자 하는, 역사와 과거부터의 맥락에 대한

요구로, 지난 1년 간 우리 동네의 버스 노선은 어떻게 변했는지, 예를 들어 특정 지역이 재개발된 후 대중교통 접근성이 어떻게 개선되었는지와 같은 시간의 흐름에 따른 변화를 이해하기에 도움을 주는 기능이다.

이러한 두 가지 상이한 데이터 접근 요구를 모두 만족시키면서도 각각의 기능이 필요한 만큼만의 데이터를 효율성과 최적화를 고려하여 서빙하기 위해, 본 프로젝트에서는 데이터 최신화를 할 때 이력 관리를 병행하여 효율적으로 수행하는 **Hot/Cold 데이터베이스 전략**을 채택하였다. 이 구조는 자주 접근하고 항상 최신 상태를 유지해야 하는 'Hot' 데이터와 변경 이력을 보존하는 'Cold' 데이터로 각각의 데이터베이스를 이원화하여 관리하는 방식이다.



①번 기능과 같이 현재 시간 중심의 조회 요구에는 **HangJeongDong**, **BusStop**, **BusData**와 같이 최신 데이터만을 담고 있는 'Hot' 모델이 즉각적으로 응답한다. 작고 정제된 최신 데이터만 보관되고 있으므로 일상적인 빈도의 요청에 빠른 응답 속도와 높은 성능을 보장할 수 있다. 반면, ②번 기능과 같이 과거 데이터의 추적 및 분석을 위해서는 각각의 'Hot' 모델에 대응하는 **HangJeongDongHistory**, **BusStopHistory**, **BusDataHistory**라는 별도의 'Cold' 모델에 모든 변경 이력을 체계적으로 보관한다. 보존 처리된 날짜와 함께 필요한 모든 과거 데이터를 체계적으로 저장함으로써 장기적인 관점에서의 연구를 수행할 수 있는 기반을 제공한다. 각 데이터 모델의 보존 시점과 방법은 데이터의 특성과 갱신 주기를 고려하여 전략적으로 다르게 구현하였다.

- BusData → BusDataHistory**

일일 단위로 수집되는 이 데이터는 변화의 폭이 크고 데이터의 양이 방대하므로, 새로운 날짜의 데이터를 가져올 때마다 기존 BusData 테이블의 모든 데이터가 BusDataHistory로 즉시 이동(아카이빙)된다. 그 후, BusData 테이블은 완전히 비워지고 새로운 데이터로 채워진다.
- HangJeongDong → HangJeongDongHistory**

이 두 데이터는 버스 승하차 정보에 비해 변경 빈도가 낮다. 따라서, 새로 받아온 데이터와 기존 데이터를 비교하여 변경이 발생했을 때, 해당 변경 사항만을 아카이빙하는 방식을 채택했다.
- BusStop → BusStopHistory**

데이터 수집 스크립트에서 API를 통해 가져온 최신 정보와 현재 데이터베이스에 저장된 데이터를 비교하여, 이름, 인구 수, 활성화 상태 등에 변경이 감지될 경우 변경이 감지된 데이터 모델 한 객체만을 Cold 데이터 모델로 옮긴다.

이러한 기능은 `utils.py`에서 각 데이터 모델 별 `fetch`, `parse`, `save` 함수 내에 논리가 구현되어 있다. Hot/Cold 분류는 서버 구동 다이어그램에 따라 데이터 최신화 명령어가 입력되어 외부 API에서 데이터를 내려받아 파싱이 완료된 후 수행된다.

3) 백엔드 주요 기능: 자동화와 최적화

지금껏 models.py에 정의된 데이터베이스 모델과 각각의 모델을 특성에 맞게 조작하기 위한 utils.py 내 함수들을 만들며 고려해야 했던 착안사항들에 대해 알아보았다. 하지만, 이렇게 데이터베이스 모델을 정의하고 수집 및 저장하기 위한 논리를 아무리 잘 만들어 놓아도 정작 실행하고 관리하지 못하면 아무런 의미가 없을 것이다. 프로젝트 개요 문단에서 전술했던 3번째 설계 중점 '지속 가능한 수집과 분석을 위한 운영 환경 구축'을 위해, 마련해 둔 여러가지 모듈과 함수들을 지휘하여 실행하는 엔진의 역할을 하는 것이 이 백엔드 파트의 모듈들이다. 이들은 정해진 시간에 맞춰 스스로 데이터를 수집, 정제, 보존하는 모든 배경 작업을 총괄하고 자동화하는 작업을 담당한다.

3-1) management/commands/

개발 초기에는 데이터 수집 · 저장과 관련된 로직이 그냥 apps.py 하나에 통째로 들어 있었지만, **추후 분석과 시각화 단계에서 코드의 재사용성을 고려**하고 가독성을 높이기 위해 API 호출, 데이터 파싱, DB 저장에 따르는 외부 API에 접근하는 공통 기능들은 utils.py 모듈에 각각의 개별 함수로 분리하였다. 프로젝트 데이터 모델 하나하나를 조작하는 개별 함수들은 utils.py에 정의되어 있지만, 이 개별 함수들을 모아 수집 · 파싱 · 저장의 순서로 실행하게 해 주는 역할을 management/commands/ 디렉토리 내 fetch_hangjeongdong_data.py, fetch_busstop_data.py 그리고 fetch_bus_data.py가 각각 맡게 되었다.

```
user % python3 manage.py fetch_hangjeongdong_data
user % python3 manage.py fetch_busstop_data
user % python3 manage.py fetch_bus_data
```

terminal

이런 방식의 모듈 분리를 통해 Django 표준 기능인 BaseCommand 기능을 활용할 수 있게 되었고, 서버 콘솔에서 관리자가 직접 명령어를 입력하여 수동으로도 미리 정의되어 있는 수집 · 파싱 · 저장의 절차를 명령어 한 줄로도 간편하게 실행할 수 있게 해주었다. 행정동이나 버스 정류장 데이터는 자치구나 관할 행정부의 사정에 따라 불규칙적인 시간에 불시에 변경사항이 발생할 수도 있는데, 이런 경우에 후술할 정기 스케줄러 작업 주기가 돌아오기 전이라도 관리자의 판단에 따라 직접 데이터 최신화가 가능하여 보다 빠르게 양질의 정보를 제공할 수 있게 되었다.

3-2) apps.py

관리자의 개입 없이도 계속해서 데이터가 최신화될 수 있어야 지속 가능한 수집과 분석을 위한 운영 환경이 구축되었다고 말할 수 있을 것이다. 바로 이 apps.py에 구현된 스케줄러를 통해 자동화된 데이터 수집 구현의 첫 단추를 꿰었다. 이 스케줄러는 정해진 시간에 맞춰 스스로 데이터를 수집하고 아카이빙하는 서버의 심장부 역할을 수행한다.

Django 애플리케이션의 생명 주기(Lifecycle) 개념 내에서 스케줄러가 작동한다. 앱이 실행 준비되었을 때 한 번 호출되는 ready() 메서드에서 Python 표준으로 제공되는 라이브러리인 threading.Timer를 이용해 스케줄러를 구현함으로써, 외부 의존성을 최소화하며 설계를 경량화할 수 있었다. 스케줄러는 데이터의 변경 주기를 고려하여, 매일 정보가 갱신되는 버스 이용량은 일일 주기로, 변동이 잦지 않은 행정동 및 버스 정류장 정보는 월별 주기로 수집하도록 설정하여 시스템 자원과 API 요청을 최적화한다.

90 [일별 데이터] 스케줄러 작업 완료. 다음 실행은 2592000초 후입니다.

데이터 주기를 고정한 이유

데이터를 받아오는 주기를 결정하는 것은 자원 효율성, 데이터 신뢰성, 그리고 서비스 요구 사항의 균형을 맞추는 데 매우 중요한 요소가 된다. 현 데이터 모델 상에서, 정류장 위치는 무료 요청 수 제한이 있는 카카오 API가 함께 사용되어 자유로운 요청에 제한 사항이 있으며, 두 가지의 API를 한 번에 사용하기 때문에 시간 소요도 크다. 데이터 양 자체도 1회에 약 1만 건 정도로, 용량은 작지만 개수만으로 받아오는 데 수 초가 걸린다. 데이터 갱신 주기를 너무 짧게 가져가게 되면 이러한 문제가 있지만, 그렇다고 너무

긴 간격으로 갱신을 하게 되면 데이터 품질과 신뢰성이 저하되고 날짜에 따른 분석 결과의 무결성을 검증하기 어려워진다. 최악의 예로 정류장이 신설되거나 폐쇄되었는데 데이터가 갱신되지 않으면 서비스는 없는 정류장을 있다고 표시하여 거짓 정보를 퍼뜨리게 되는 경우를 들 수 있겠다.

이러한 문제에 대해 두 가지 방식을 선택할 수 있었다:

- 고정된 주기로 데이터를 받는다. (예: 매주, 매월 특정 날짜...)
- 어차피 매일 최신화하는 승하차 정보 데이터 상에서, 이전에 없던 정류장 ID가 발견되거나 특정 데이터 패턴의 변화가 감지되면 정류장과 행정동의 정보에 변화가 있다고 가정하고 그 때 되어서야 데이터를 받는다.

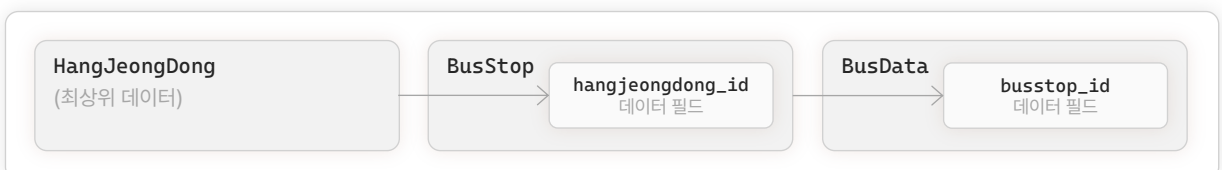
그러나, 승하차 정보에 따라 동적으로 최신화 주기를 결정하려면, 매일 4만 건이 넘게 쌓이는 데이터들의 ID를 전부 다 기록하는 논리와 데이터 모델이 별도로 존재해야 하며, 간혹 서울시 API 자체에서 특정 정류장 정보를 일시적으로 누락하는 경우도 있어 현재 주어진 한정된 자원 내에선 모기 잡으려 총 쏘는 격이 될 가능성이 높아 보였다. 한편 고정 주기로 데이터를 가져오면, 다음 데이터 최신화 주기 사이에 데이터가 변경된다면 최신성을 잃는 것이 아닌가 하는 우려가 있었다. 하지만 매일 가져오는 승하차 인원 모델을 제외한 나머지 두 모델, 정류장 위치와 행정동 인구 데이터는 하루아침에 큰 범위로 바뀌는 경우가 드물며, 특히 버스 정류장 한 두 개의 데이터가 누락된다고 하여 특정 행정동의 교통 접근성이라는 거시적인 지표가 극적으로 변하기는 쉽지 않다는 사실까지 고려하여 고정 주기라는 차선택을 선택하게 되었다. 버스 정류장 정보는 공사나 시위로 인한 노선 조정 공지가 보통 5영업일 단위로 나온다는 점을 고려하여 일주일로 설정했고, 행정동 데이터는 정류장보다 조금 더 변동성이 낮고, 통계청 등에서 발표하는 실효성 있는 공식적인 인구 데이터가 월별, 분기별로 집계되어 나옴을 고려하여 1개월 간격으로 설정하여 충분히 어느 정도의 최신성을 확보할 수 으면서, 안정적인 데이터 운영이 가능한 방향으로 기능을 설계하였다.

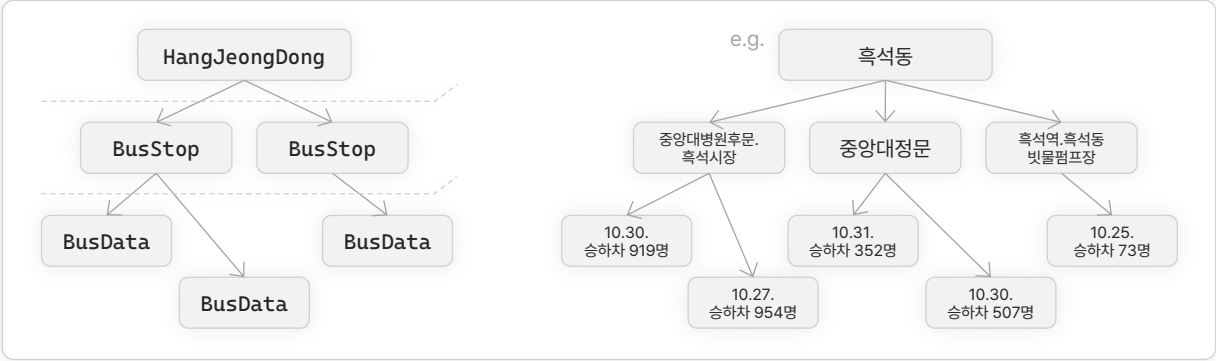
4) 프론트엔드 주요 기능: 데이터 조회

열심히 수집한 데이터들을 사용자에게 보여주기 위하여, 그리고 서비스의 핵심 데이터인 '행정동'과 '버스 정류장', 그리고 '승하차 인원'까지의 포함 관계를 명확하게 표현하고 향후 기능 확장에 유연하게 대처할 수 있도록 초기 설계 단계에서부터 REST 아키텍처 원칙을 준수하고자 노력하였다. 사용자 입장에서 서버로의 첫 발걸음이 되는 URL은 다음과 같이 설계되어 있다:

/	최상위 진입점 (index.html): 서비스의 첫 화면으로, 사용자가 데이터를 탐색할 최상위 자원인 행정동을 선택하는 인터페이스를 제공한다.
/hangjeongdong/(행정동 코드)	행정동 데이터의 표시 (hangjeongdong_detail.html): 최상위 진입점에서 행정동을 선택하면 표시되며, 해당 행정동의 메타데이터와 소속되어 있는 정류장 목록을 제공한다.
/hangjeongdong/(행정동 코드)/busstop/(정류장 코드)	정류장 데이터의 표시 (busstop_detail.html): 선택한 정류장의 메타데이터와 세부적인 승하차 통계 정보가 표시된다.

본 서비스에서는 정해진 데이터를 정해진 위치에서 받아와 보여주는 기능이 중심이므로, URL과 views를 설계할 때 사용자가 직접 정보를 수정하는 POST나 메서드나 forms 구현은 운영 목적에 부합하지 않아 제외하고, 대신 GET 메서드에서 직관적으로 탐색할 수 있는 구조를 중심으로 설계하였다. 전체적인 URL의 구조에서 알 수 있듯, models.py에서 데이터 모델을 정의할 때 고려하여 만들었던 트리 구조를 그대로 가져왔다. 가장 큰 데이터 범위인 '행정동'부터 시작하여 > 행정동 내에 분포한 '버스 정류장' > 버스 정류장 별 '승하차 인원수'까지 이어지는 부모-자식 관계에서, 각각의 일련 번호에 따른 특정 행정동이나 정류장 하나를 식별하기 위해 **탐색 흐름이 주소창에 바로 보이는 URL 파라미터 형태로** 구현하였다.





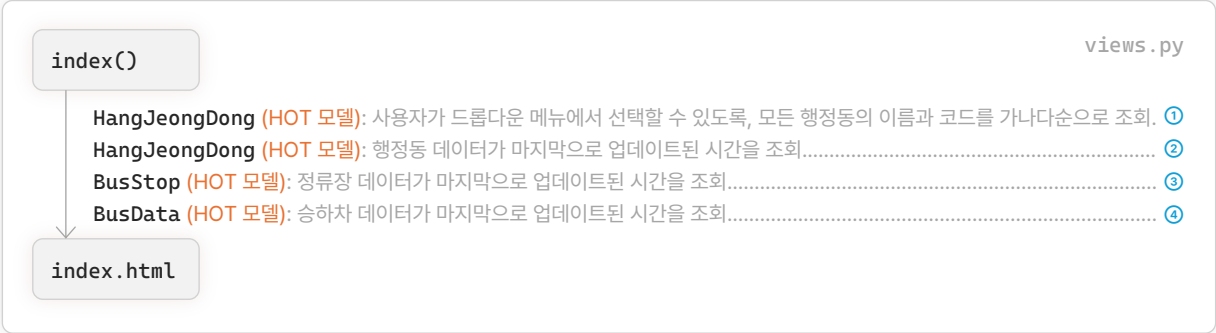
이러한 트리 구조 기반의 URL 파라미터형 탐색에 더하여, 버스 정류장이 담고 있는 수많은 승하차 데이터를 **일자별로 사용자의 요구에 맞게 동적으로 필터링하기** 위해 쿼리 파라미터를 적용하였다.

```
/hangjeongdong/{행정동 코드}/busstop/{정류장 코드}/{필터링 날짜 선택 쿼리 구분}
```

버스 정류장 상세 페이지에서 특정 범위의 날짜만 걸러 조회하는 기능을 이와 같은 구조로 접근할 수 있도록 구현하였다. 이처럼 URL 파라미터로 고유한 속성을 가진 자원을 식별하고, 쿼리 파라미터로 데이터의 표현을 제어하는 역할을 용도에 맞게 분리하여 함께 사용함으로써, 누가 사용하더라도 그 의도를 쉽게 파악할 수 있는 예측 가능하고 직관적인 구조를 갖추고자 하였다.

4-1) 프론트엔드와 데이터베이스

URL 원칙으로 사용자가 서버에게 무엇을 원하는지를 구조적으로 전달하는 약속을 정의했다. 그렇다면, 서버는 이렇게 정의된 요청을 받은 뒤, 어떻게 동작하여 사용자에게 최종 화면을 보이게 될까? 아래는 Django의 MVT 패턴에 따라 서버 내부의 View가 Model(데이터베이스)와 상호작용하여 데이터를 가공하고, 최종적으로 Template(HTML)을 사용자에게 전달하는 흐름도이다:



hangjeongdong_detail()

views.py

- HangJeongDong (HOT 모델): 행정동 개별 요약 정보 조회..... ①
- BusData (HOT 모델): 전체 승하차 데이터 중 가장 마지막 날짜가 언제인지 확인..... ②
- BusStop (HOT 모델): 현재 URL의 행정동 코드에 해당하는 모든 정류장 목록을 이름순으로 조회..... ③
- BusData (HOT 모델): 위 두 절차에서 찾은 최신 날짜와 정류장 목록을 조건으로 각 정류장별 최신 날짜 하루의 승하차 인원 집계. ④

hangjeongdong_detail()

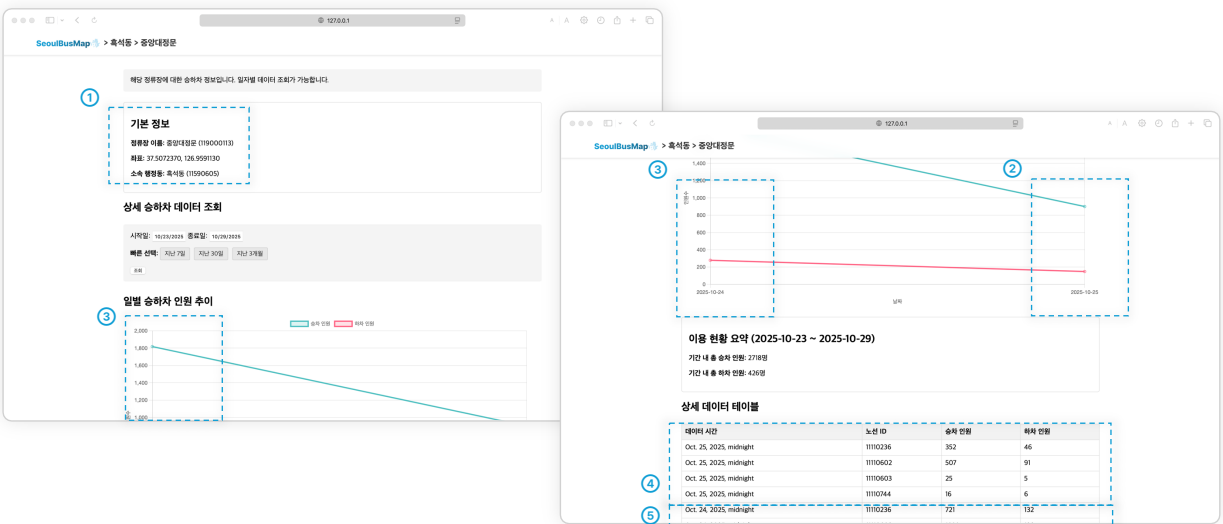


busstop_detail()

views.py

- BusStop (HOT 모델): 버스 정류장 개별 요약 정보 조회..... ①
- BusData (HOT 모델): 사용자가 요청한 최신 날짜의 일별 승하차 인원 합계 조회..... ②
- BusDataHistory (COLD 모델): 사용자가 요청한 기간 내 과거 날짜의 승하차 인원 합계 조회..... ③
- BusData (HOT 모델): 사용자가 요청한 최신 날짜의 모든 개별 승하차 기록 조회..... ④
- BusDataHistory (COLD 모델): 사용자가 요청한 기간 내 과거 날짜의 모든 승하차 기록 조회..... ⑤

busstop_detail()



위와 같은 과정을 통해 각각의 주소의 목적에 맞는 내용을 화면에 표시한다. index.html의 메인 페이지는 사용자가 분석을 시작할 최상위 자원인 행정동을 선택하게 하는 입구의 역할을 수행한다. 행정동 상세 페이지는 해당 지역의 전반적인 활동성을 빠르게 파악할 수 있는 중간 대시보드 역할을 한다. 이 페이지의 주요 기능인 '정류장별 최신 승하차 인원' 표시는 '지금 가장 활발한 곳이 어디인가' 같은 일상적으로 가벼운 질문에 즉각적으로 답을 주기 위한 것이다. 따라서 이처럼 빠르고 빈번하고 많은 최신 현황 조회가 필요한 기능을 위해, 이 페이지는 Hot 데이터셋을 신속하게 조회하도록 설계하였다. 버스 정류장 상세 페이지는 사용자가 상세 데이터를 통해 특정 기간의 추세를 살펴볼 수 있는, 서비스가 제공하는 범위에서의 심층 분석의 종착점이다. 이 페이지는 과거 데이터의 완전성이 무엇보다 중요하므로, 역사적 데이터에 대한 분석을 가능케 하기 위해 Hot 데이터셋과 Cold 데이터셋을 모두 통합하여 조회함으로써 사용자가 원하는 어떤 기간에 대해서도 데이터의 누락 없이 정확한 통계를 얻을 수 있도록 구현하였다.

이처럼 각 페이지의 기능적 요구사항에 맞춰 데이터 접근 방식을 달리하는 선택을 통해, 가벼운 요청에는 빠른 응답성을, 깊이 있는 분석에는 데이터의 정확성에 포커스를 맞추어 성능과 신뢰도 두 마리 토끼를 모두 잡는 최적화를 달성해내고자 하였다. 이러한 프론트엔드 인터페이스와 데이터 접근 패턴에 대한 설계로 사용자 친화적이고 효율적이며 확장 가능한 데이터 탐색 서비스를 완성하였다.

3. 결론

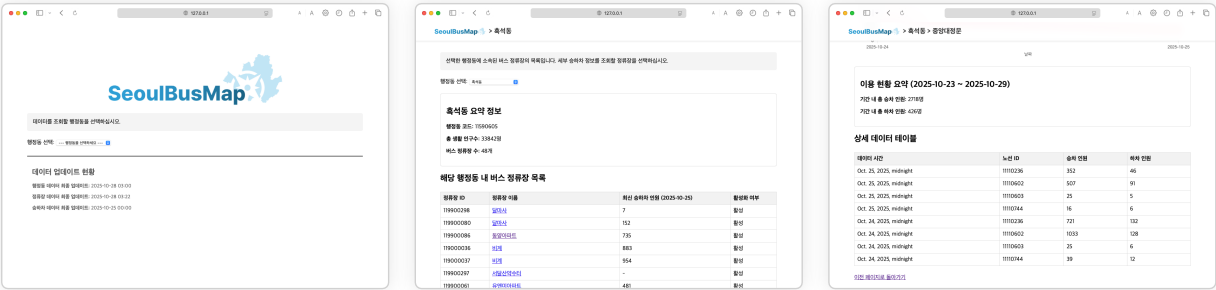
1) 프로젝트 성과 및 데이터 수집 계획

지금까지 서울시 대중교통 이용률 증가와 지역 간 인구 수 격차 문제 등을 배경으로 시민 생활의 필수 인프라인 버스 서비스 공급의 지역별 불균형 문제를 진단하는데 목적을 둔 SeoulBusMap의 설계에 대해 서술하였다. 현재까지 구축한 파이프라인에 따른 초기 데이터 수집 성과는 다음과 같이 정리할 수 있다.

- **인구 데이터:** HangJeongDong 모델에는 행정동별 최신 평균 생활 인구수를 저장하였다. 이 데이터는 버스 서비스 공급의 적정성을 측정하는 데 있어 가장 객관적이고 필수적인 기준 역할을 수행하며 행정동별 생활 인구 통계는 단순한 주민등록 인구가 아닌, 실제 도시 내에서 활동하는 인구 수요를 반영하기 때문에 서비스 공급이 지역 주민의 실질적인 교통 수요에 기반하여 심층적으로 분석할 수 있는 토대를 만들었다.
- **버스 공급 인프라:** BusStop 모델에는 서울시 모든 버스 정류장의 위치 좌표와 활성화 상태 정보를 확보하였다. 이는 곧 공간 분석의 핵심 기반이 완성되었음을 의미하며, 특히 우리는 정류장의 좌표 데이터를 활용하여 카카오 Local REST API를 통해 각 정류장이 속한 행정동 코드(district_id)와의 매핑을 성공적으로 완료했다. 이로써 버스 정류장과 행정동 구역을 공간적으로 통합하는 분석 환경을 구축하여 인구 기반의 정밀한 서비스 공급 적정성 평가가 가능해졌다.
- **버스 이용 실태:** 버스 노선별 및 정류장별 일일 승하차 인원 데이터를 BusData 모델에 시간의 흐름에 따라 순차적으로 기록하고 누적하고 있다. 이는 단순히 정류장의 개수 같은 공급량 지표를 넘어서, 실제 시민들의 버스 이용 행태와 수요를 반영하는 실증적인 지표 역할을 한다. 이 이용률 데이터를 활용하여 버스 서비스가 시민들의 교통 수요를 얼마나 충족시키고 있는지 효율성 측면에서 다각적으로 평가할 수 있는 기반을 마련하였다.

현재 설계가 확정된 데이터 모델에 성공적으로 10월 후반부의 데이터를 이미 저장해 둔 상태이다. 본 보고서와 함께 제출된 프로젝트 코드 파일에 데이터베이스가 포함되어 있으며, runserver에서 서버를 구동하면 로컬 환경에서도 데이터 열람이 가능하다. 가짓수는 많지만, 아직 용량이 크진 않아 업로드가 가능한 수준이다. 흠어진 오픈API 데이터를 아카이빙하는 본 프로젝트의 특성과 함께 전술한 자동화된 스케줄러 구현 덕분에 원격, 로컬 서버 환경과 무관하게 데이터 수집은 서버 실행만 해 두면 자동으로 이루어져 큰 제한 사항 없이 가능할 것으로 보이며, 보고서 작성 시간 기준 10월 25일의 최신화 스케줄이 돌아와 현재도 수집이 진행 중이다. 수집을 위한 서버 구동은 팀원 자체 장비를 통한 로컬 실행으로 처리할 방침이며, 이후 프로젝트 진행 중 타 기기에서의 이용이 필요해질 것을 대비하여 클라우드 플랫폼으로의 이동을 고려하는 중이다.

아래는 실제 본 프로젝트의 서버가 로컬에서 가동된 상태에서 수집된 데이터가 프론트엔드 페이지로 출력되고 있는 스크린샷이다.



2) 향후 발전 방향

성공적인 데이터 확보에도 불구하고, 프로젝트의 분석적 결론을 더욱 정교화하고 확장하기 위해서는 현 데이터셋이 가진 본질적인 한계점을 명확히 인지해야 할 필요가 있다고 판단했다. 현재 시점에서 한계점으로 들 수 있는 부분은 행정동별 생활 인구를 기준으로 사용하고 있는데, 상업 및 업무 지구의 경우 주간 유동인구가 과도하게 반영되어 순수 거주민의 교통 소외 문제를 도출하지 못할 가능성이 있다. 이를 해결하기 위해 요일별 가중치를 적용하여 수집된 데이터가 특정 요일(e.g. 주말)의 특성에 과도하게 영향 받는 것을 방지하기 위해 주 7일 데이터의 평균을 사용하거나 요일별 변동성을 반영하여 분석 결과의 신뢰성을 높이는 다양한 방향을 검토 중이다. 또한 우리는 일일 총 승하차 인원만을 기준으로 분석하고 있는데, 출퇴근 시간에 승하차 인원이 몰리는 현상이나 심야시간의 실제 서비스 이용 편의성 격차를 정확하게 파악하기 어렵다는 문제가 있다. 이 문제는 시간대별 평균 승하차 인원수를 제공하는 API의 이용 등으로 해결을 검토하고 있다.

중간고사까지 본 프로젝트의 설계 범위는 '수집 및 아카이빙'으로, 추후 분석과 시각화를 구현하기 위해 필요할 것으로 예상하고 만들어 두었지만 현 단계에서는 사용하지 못한 데이터 모델과 필드가 있다. BusData를 제외한 Cold 데이터셋 전체, 승하차 데이터 모델에 포함되어 있는 노선 정보를 담은 'bus_id' 필드, 행정동과 정류장 데이터 모델의 'timestamp' 필드가 그 예시인데, 분석 및 시각화 구현 시 트리 구조 단에서의 시간대별 조회 기능과 인기/비인기 노선 조회 등에 응용할 예정이다.

이상으로 서울시 공공데이터를 안정적으로 수집 및 관리하는 백엔드 시스템과, 수집된 데이터를 효과적으로 탐색할 수 있는 웹 인터페이스를 구축하는 것을 목표로 중간 프로젝트를 진행하였다. 체계적인 데이터 모델링, Hot/Cold 아카이빙 전략, 그리고 RESTful 원칙에 입각한 API 설계를 통해, 향후 다양한 데이터 분석을 수행하기 위한 기술적 기반을 마련하였다.

본 프로젝트는 이후 이어질 과제의 주제인 '데이터의 심층 분석 및 시각화'를 수행하기 위한 필수적인 선행 단계이다. 향후 서비스에서 수집한 정제된 데이터를 바탕으로, 본격적으로 행정동별 인구수 대비 버스 공급의 적절성을 분석하고, 지도 기반의 시각화 및 통계 차트 고도화를 통해 교통 소외 지역을 도출하는 등, 보다 구체적이고 의미 있는 결과를 만들어 나갈 계획이다.

*서버 실행 시 간혹 서울시 API 데이터 패치에 오류가 발생하는 경우가 있습니다.

이는 일시적으로 서울시에서 필드값이 누락된 데이터를 보내 준 경우로, 명령어를 재시작하면 정상 작동합니다.



서울시 교통 소외 지역을 조명하는 공익 데이터 수집 · 분석 서비스

중앙대학교 예술공학대학 예술공학부

김도균 (20225525)

김도연 (20232782)

유형준 (20230749)

이지민 (20236743)